Name: Y.Veerendar Reddy **Reg.No:-** 192325102

Subject Code/Name: MLA0305-Reinforcement Learning

EXPERIMENT – 8

Implementation and Comparative Analysis of SARSA and Q-Learning

Aim

To implement SARSA and Q-Learning algorithms and compare their learning performance using a simple Reinforcement Learning environment.

Procedure

1. Define the states, actions, rewards, and transitions.
2. Initialize the Q-tables for SARSA and Q-Learning.
3. Select actions using an ε-greedy strategy.
4. Update the Q-values using the SARSA and Q-Learning equations.
5. Repeat the learning process for several episodes.
6. Calculate the cumulative rewards.
7. Compare the learning performance of both algorithms using graphs.

Code:

```
# ============================================================
# EXPERIMENT 8: SARSA vs Q-LEARNING
# ============================================================

import numpy as np
import matplotlib.pyplot as plt

np.random.seed(42)

# States: Start -> Middle -> Goal
n_states = 3
n_actions = 2

# Actions: 0 = Move, 1 = Stay
Q_sarsa = np.zeros((n_states, n_actions))
```

```
Q_qlearn = np.zeros((n_states, n_actions))

alpha = 0.1
gamma = 0.9
epsilon = 0.1
episodes = 200

# Reward function
def get_reward(state, action):
    if state == 0 and action == 0:
        return 1
    if state == 1 and action == 0:
        return 10
    if action == 1:
        return -1
    return 0

# Transition function
def transition(state, action):
    if state == 0:
        return 1 if action == 0 else 0
    if state == 1:
        return 2 if action == 0 else 1
    return 2

# Epsilon-greedy action
def choose_action(Q, state):
    if np.random.random() < epsilon:
        return np.random.randint(n_actions)
    return np.argmax(Q[state])

sarsa_rewards = []
qlearn_rewards = []

# ============================================================
```

```
# SARSA
# ============================================================

for ep in range(episodes):

    state = 0
    action = choose_action(Q_sarsa, state)
    total = 0

    while state != 2:

        next_state = transition(state, action)
        reward = get_reward(state, action)
        next_action = choose_action(Q_sarsa, next_state)

        # SARSA update
        Q_sarsa[state, action] += alpha * (
            reward +
            gamma * Q_sarsa[next_state, next_action]
            - Q_sarsa[state, action]
        )

        total += reward
        state, action = next_state, next_action

    sarsa_rewards.append(total)

# ============================================================
# Q-LEARNING
# ============================================================

for ep in range(episodes):

    state = 0
    total = 0
```

```
    while state != 2:

        action = choose_action(Q_qlearn, state)
        next_state = transition(state, action)
        reward = get_reward(state, action)

        # Q-Learning update
        Q_qlearn[state, action] += alpha * (
            reward +
            gamma * np.max(Q_qlearn[next_state])
            - Q_qlearn[state, action]
        )

        total += reward
        state = next_state

    qlearn_rewards.append(total)

# ============================================================
# RESULTS
# ============================================================

print("=" * 50)
print("SARSA Q-TABLE")
print("=" * 50)
print(Q_sarsa)

print("\nQ-LEARNING Q-TABLE")
print("=" * 50)
print(Q_qlearn)

print("\nAverage SARSA Reward:",
      np.mean(sarsa_rewards))
```

```
print("Average Q-Learning Reward:",
    np.mean(qlearn_rewards))

print("\nSARSA Best Actions:")
print(np.argmax(Q_sarsa, axis=1))

print("\nQ-Learning Best Actions:")
print(np.argmax(Q_qlearn, axis=1))

# ============================================================
# VISUALIZATION
# ============================================================

plt.figure(figsize=(10, 5))

plt.plot(
  sarsa_rewards,
  label="SARSA"
)

plt.plot(
  qlearn_rewards,
  label="Q-Learning"
)

plt.title("SARSA vs Q-Learning")
plt.xlabel("Episode")
plt.ylabel("Reward")
plt.legend()
plt.grid(True)

plt.show()
```

OUTPUT:-

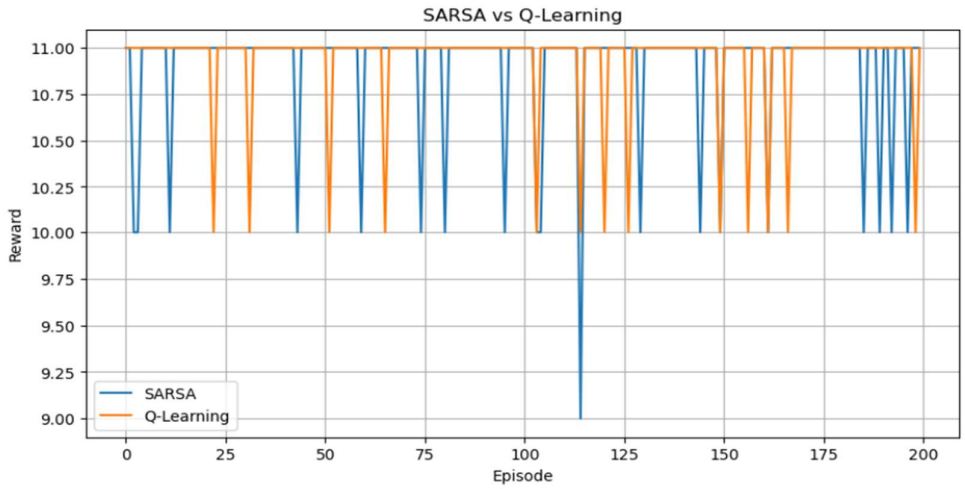
```
================================================
SARSA Q-TABLE
================================================
[[9.45235922 3.64819946]
 [9.99999999 5.48928043]
 [0.         0.        ]]

Q-LEARNING Q-TABLE
================================================
[[9.99999985 2.54441314]
 [9.99999999 4.88333586]
 [0.         0.        ]]

Average SARSA Reward: 10.9
Average Q-Learning Reward: 10.935

SARSA Best Actions:
[0 0 0]

Q-Learning Best Actions:
[0 0 0]
```



Visualization

The graph compares the reward obtained by SARSA and Q-Learning over multiple episodes.

- **SARSA** is an on-policy algorithm because it updates using the action actually selected by the current policy.
- **Q-Learning** is an off-policy algorithm because it updates using the maximum estimated Q-value of the next state.

The graph helps compare how quickly the two algorithms learn.

Summary

SARSA and Q-Learning were implemented using Q-tables and ε-greedy action selection. Both algorithms learned the best actions through repeated interaction with the environment.

Result

Thus, SARSA and Q-Learning were successfully implemented and compared, and their learning performance was visualized using episode-wise rewards.